

Spark & SQL Temelleri

Junior Data Engineer İçin — Kod Hücreleri Üzerinden

EnergyLens battery notebook'larından (notebooks/_deferred/page9-battery/) gerçek kod

Bu doküman bir mülakat pozisyonlama rehberi değildir. Tek amacı: bir junior data engineer adayından beklenen Spark ve SQL temellerini, uydurma örnekler yerine senin gerçekten yazdığın kod hücreleri üzerinden göstermek. Her başlık aynı düzende: kavram → gerçek kod → (varsa) SQL karşılığı → kısa not.

BÖLÜM 1 — SQL Temelleri

1.1 Filtreleme (WHERE) ve kolon türetme

Junior standardı — her adayın kör gözle yazabilmesi beklenir.

Bir DataFrame'i satır bazında filtrelemek ve yeni bir kolon türetmek, SQL'deki WHERE ve SELECT ... AS kadar temel. Aşağıda ham 15 dakikalık batarya okumaları önce kapsamdaki binalara filtreleniyor, sonra timestamp'ten bir 'date' kolonu türetiliyor.

```
df_battery_daily = (  
    df_battery_raw  
    .filter(F.col("building_id").isin(SCOPE_BUILDINGS))  
    .withColumn("date", F.to_date("timestamp_utc"))  
)
```

SQL karşılığı:

```
SELECT *, CAST(timestamp_utc AS DATE) AS date  
FROM bronze_battery_status  
WHERE building_id IN ('B001', 'B003', 'B004', 'B005', 'B006', 'B007')
```

1.2 GROUP BY + aggregation (+ CASE WHEN)

Junior standardı — SUM/AVG/COUNT + koşullu toplama her mülakatta çıkar.

Günlük toplama: bina ve tarihe göre grupla, ortalama SoC al, sadece pozitif şarj gücünü enerjiye çevirip topla (F.when = SQL'deki CASE WHEN), satır say.

```
df_battery_daily = (
    df_battery_raw
    .groupBy("building_id", "date")
    .agg(
        F.avg("soc_raw").alias("avg_soc_raw"),
        F.sum(
            F.when(F.col("charge_power_raw") > 0,
                F.col("charge_power_raw") * (15.0/60.0))
            .otherwise(0)
        ).alias("charge_kwh_raw"),
        F.count("*").alias("readings_count"),
    )
)
```

SQL karşılığı:

```
SELECT building_id, date,
       AVG(soc_raw) AS avg_soc_raw,
       SUM(CASE WHEN charge_power_raw > 0
                THEN charge_power_raw * (15.0/60.0) ELSE 0 END) AS charge_kwh_raw,
       COUNT(*) AS readings_count
FROM bronze_battery_status
GROUP BY building_id, date
```

1.3 JOIN (LEFT JOIN) — fact tabloyu referans tabloyla zenginleştirme

Junior standardı — INNER vs LEFT farkını net açıklayabilmelisin.

Bina tablosunu (fact'e yakın) ülke bazlı fiyat referans tablosuyla birleştiriyoruz. LEFT JOIN kullanılıyor çünkü fiyatı olmayan bir ülke yüzünden bina satırını KAYBETMEK istemeyiz — INNER JOIN bunu sessizce silerdi.

```
df_bldg_config = (
    df_building_p9.select(
        "building_id", "building_name", "country_code", "has_battery"
    )
    .join(
        df_pricing,
        df_building_p9["country_code"] == df_pricing["country"],
        "left"
    )
)
```

SQL karşılığı:

```
SELECT b.building_id, b.building_name, b.country_code, b.has_battery, p.*
FROM silver_building_master b
LEFT JOIN ref_electricity_tariffs p
ON b.country_code = p.country
```

Not: kodda gerçek satırda F.broadcast(df_pricing) de var — bu SQL'in değil, Spark'ın performans katmanının konusu, Bölüm 2.4'te ayrı işleniyor.

1.4a Pencere fonksiyonu — 'her grubun en son satırı'

Bonus / orta seviye sinyali — çoğu junior aday bunu ezbere bilmez, sen biliyorsun.

Fiyat tablosunda ülke başına birden fazla yıl satırı var. Her ülke için sadece EN GÜNCEL yılı almak için ROW_NUMBER mantığı: yıla göre sırala, ilkinin (rn=1) al.

```
_tar_latest = (\n    spark.table("ref_electricity_tariffs")\n    .withColumn("_rn", F.row_number().over(\n        Window.partitionBy("country_code").orderBy(F.col("year").desc())\n    ))\n    .filter(F.col("_rn") == 1)\n)
```

SQL karşılığı:

```
SELECT * FROM (\n    SELECT *, ROW_NUMBER() OVER (\n        PARTITION BY country_code ORDER BY year DESC) AS rn\n    FROM ref_electricity_tariffs\n) WHERE rn = 1
```

1.4b Pencere fonksiyonu — yürüyen (kümülatif) toplam

Bonus / orta seviye sinyali — running total / year-to-date mantığı.

Her bina için, ilk günden bugüne kadar birikmiş toplam tasarrufu hesaplamak: aynı ROW_NUMBER ailesinden ama bu sefer SUM ... OVER, sınırsız-geçmişten-bugüne aralığıyla (rowsBetween unboundedPreceding, 0).

```
window_cum = (\n    Window.partitionBy("building_id")\n    .orderBy("date")\n    .rowsBetween(Window.unboundedPreceding, 0)\n)\ndf_daily_summary = df_best_strategy.withColumn(\n    "total_cost_avoided_eur",\n    F.sum("net_savings_eur").over(window_cum)\n)
```

SQL karşılığı:

```
SELECT *,\n    SUM(net_savings_eur) OVER (\n        PARTITION BY building_id ORDER BY date\n        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW\n    ) AS total_cost_avoided_eur\nFROM best_strategy_per_day
```

Aynı araç (pencere fonksiyonu), iki farklı iş sorusu: 1.4a 'en güncel satır', 1.4b 'yürüyen toplam'. Mülakatta bu ayrımı kurabilmen güçlü bir sinyal.

1.5 NULL yönetimi (COALESCE / GREATEST / LEAST)

Junior standardı — 'kirli veriyle nasıl başa çıkarsın' sorusunun cevabı.

Sensör verisi null ya da hatalı negatif olabilir. COALESCE eksik değere varsayılan veriyor, GREATEST negatif gürültüyü sıfıra kırıyor. Bunu ham veri finansal hesaba girmeden HEMEN yapmak önemli — sona bırakılırsa yanlış ama 'makul görünen' bir sonuç üretir.

```
.withColumn(  
  "charge_kwh",  
  F.greatest(F.lit(0.0), F.coalesce(F.col("charge_kwh_raw"), F.lit(0.0)))  
)
```

SQL karşılığı:

```
SELECT GREATEST(0.0, COALESCE(charge_kwh_raw, 0.0)) AS charge_kwh  
FROM ...
```

BÖLÜM 2 — Spark Temelleri

2.1 SparkSession ve konfigürasyon

Junior standardı — her notebook'un ilk hücresi budur.

SparkSession, Spark kümesine açılan tek giriş kapısı. AQE (Adaptive Query Execution) sorgu planını çalışma sırasında optimize eder; shuffle.partitions join/groupBy sonrası verinin kaç parçaya bölüneceğini belirler.

```
spark = SparkSession.builder.getOrCreate()  
spark.conf.set("spark.sql.adaptive.enabled", "true")  
spark.conf.set("spark.sql.shuffle.partitions", "16")
```

2.2 Transformation vs. action — lazy evaluation

Junior standardı — Spark'ın en çok sorulan temel kavramı.

.withColumn() zinciri hiçbir şeyi HEMEN çalıştırmaz — sadece bir plan (DAG) kurar. Bunlara 'transformation' denir. .count() gibi bir 'action' çağrılana kadar hiçbir iş fiilen yapılmaz. Buna 'lazy evaluation' denir.

```
df_dispatch = (  
  df_joined  
    .withColumn("battery_type_clean", F.when(...).otherwise("LFP"))  
    .withColumn("charge_kwh", F.greatest(F.lit(0.0), ...))  
    .withColumn("net_savings_eur", F.col("cost_avoided_eur") - ...) )  
# Buraya kadar HİÇBİR ŞEY çalışmadı — sadece plan kuruldu.  
print(f"Dispatch rows computed: {df_dispatch.count():,}")  
# .count() bir ACTION -> plan burada gerçekten çalışır.
```

2.3 Şema: StructType ile zorlama vs. inferSchema ile tahmin

Junior standardı — 'veri kalitesini nasıl sağlarsın' sorusunun temel parçası.

Kendi ürettiğin veri için şemayı EL İLE tanımla (StructType/StructField) — tip garantisi verir. inferSchema'yı SADECE güvenilmeyen ham bir CSV'yi okurken, fallback olarak kullan; Spark'ın tip TAHMİN etmesine üretim tablosunda güvenme.

```
batt_tech_schema = StructType([
    StructField("battery_id", StringType(), True),
    StructField("capacity_kwh", DoubleType(), True),
    StructField("eu_compliant", BooleanType(), True),
])
df_battery_tech = spark.createDataFrame(rows, schema=batt_tech_schema)

# --- sadece ham CSV fallback okurken ---
spark.read.format("csv").option("inferSchema", "true").load(csv_path)
```

2.4 Broadcast join — küçük referans tablosunu her worker'a kopyala

Bonus / orta seviye sinyali — performans farkını AÇIKLAYABİLMEK önemli.

df_pricing ülke başına 1 satır — çok küçük. broadcast() bu küçük tabloyu her worker'a kopyalar; Spark büyük dispatch verisini ağ üzerinden karıştırmak (shuffle) zorunda kalmaz. Küçük-dimension + büyük-fact join'lerinde standart.

```
df_bldg_config = df_building_p9.join(
    F.broadcast(df_pricing),
    df_building_p9["country_code"] == df_pricing["country"],
    "left"
)
```

SQL karşılığı:

```
-- Spark SQL'de karşılığı bir join hint'i:
SELECT /*+ BROADCAST(p) */ b.*, p.*
FROM building_master b LEFT JOIN pricing p ON b.country = p.country
```

2.5 UDF (User Defined Function) — ne zaman ve neden dikkatli kullanılır

Junior standardı bilmek, bonus olan MALİYETİNİ açıklayabilmek.

NPV/IRR hesabı (Newton-Raphson iterasyonu) için native bir Spark SQL fonksiyonu yok — bu yüzden Python fonksiyonunu udf() ile Spark'a tanıtıyoruz. Bedeli: her satır Python'a serialize edilip geri alınır, Catalyst optimizier devre dışı kalır — büyük veride yavaştır. Burada veri küçük (bina x strateji, ~24-40 satır) olduğu için bu maliyet önemsiz.

```
def compute_irr(annual_savings, total_capex, ...):
    # Newton-Raphson iterasyonu; native SQL karşılığı yok
    ...
    return r * 100.0

irr_udf = udf(lambda s, c: compute_irr(s, c) if s and c else None, DoubleType())
df_simulation = df_sim_base.withColumn(
    "irr_percent",
    irr_udf(F.col("annual_savings_eur").cast(DT()), F.col("total_capex_eur").cast(DT()))
)
```

Mülakatta söyle: 'UDF'lerin milyonlarca satırlık sıcak yolda doğru desen olmadığını biliyorum; orada native Spark ifadesi ya da vectorized pandas UDF kullanırım. Burada veri hacmi bunu önemsiz kıldı.'

2.6 Ne zaman Spark'a gerek YOK — küçük veri için düz Python/pandas

Bonus / orta seviye sinyali — 'her şeye Spark' refleksinin YOKLUĞU olgunluk gösterir.

96 satırlık (4 strateji x 24 saat) bir referans tablo için bunu dağıtık motora sokmanın hiçbir faydası yok. Düz Python/pandas ile satırları kur, Spark'ı SADECE en sonda, Delta'ya kalıcı yazarken devreye sok.

```
rows = []
for strategy_key, (label, profile) in strategies_data.items():
    for hour in range(24):
        rows.append({...})
pdf = pd.DataFrame(rows)

# Spark SADECE burada devreye giriyor -> Delta yazımı için
sdf = spark.createDataFrame(pdf, schema=schema)
sdf.write.format("delta").mode("overwrite") \
    .saveAsTable("gold_battery_hourly_profile")
```

2.7 Delta Lake: overwrite vs. MERGE (upsert), şema evrimi

Junior standardı bilmek, bonus olan gerçek bir trade-off ile savunabilmek.

'overwrite' tabloyu her çalıştırmada BAŞTAN yazar — basit ve idempotent (tekrar çalıştırınca aynı sonucu verir). 'MERGE' (upsert) sadece değişen satırları günceller — büyük/artımlı veride daha verimlidir ama YENİ KOLON eklenince güvenilir olmayabilir. Bu tabloya sürekli yeni kolon (battery health, cycle sayısı) eklendiği için bilinçli olarak overwrite'a geçilmiş.

```
# 2026-06-06: OVERWRITE (MERGE değil) -> yeni kolonlar her çalıştırmada
# güvenilir şekilde kalsın diye.
df_dispatch.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("gold_battery_dispatch")
```

2.8 Idempotency (tekrar çalıştırılabilirlik)

Junior standardı — klasik mülakat sorusu: 'pipeline'in idempotent mi?'

2.7'deki overwrite modunun doğrudan sonucu: notebook'u art arda iki kez çalıştırmak AYNI tabloyu üretir, satırları çoğaltmaz. Alternatif olan append modu bunu garanti etmez — elle bir dedup/silme mantığı gerektirir.

```
# İki kez çalıştır -> sonuç aynı (idempotent):
df.write.format("delta").mode("overwrite").saveAsTable("gold_battery_dispatch")

# Riskli alternatif (append) -> iki kez çalıştıırırsan satırlar İKİYE katlanır:
# df.write.format("delta").mode("append").saveAsTable("gold_battery_dispatch")
```

2.9 Yazmadan önce ucuz veri-kalite kontrolleri (assert)

Junior standardı — 'veri kalitesini nasıl sağlarsın' sorusuna somut kanıt.

Bir test paketi değil ama neredeyse sıfır maliyetle koca bir hata sınıfını yakalıyor: beklenen satır sayısı tutuyor mu, hiçbir tasarruf rakamı negatif değil mi (negatiflik bir yerde işaret/formül hatası olduğunu gösterir).

```
assert sdf.count() == len(BUILDINGS) * len(STRATEGIES), "row count mismatch"
assert sdf.filter("annual_savings_eur < 0").count() == 0, "negative savings"

sdf.write.format("delta").mode("overwrite").saveAsTable("gold_battery_simulation")
```

BÖLÜM 3 — Junior Data Engineer'dan Beklenen — Öz Kontrol Listesi

Kapatmadan önce, notlara bakmadan, her satırı 1-2 cümleyle kendi kendine açıklayabildiğini kontrol et:

- SQL: WHERE ile filtreleme ve SELECT ... AS ile kolon türetme (1.1)
- SQL: GROUP BY + SUM/AVG/COUNT, ve CASE WHEN ile koşullu toplama (1.2)
- SQL: INNER JOIN ile LEFT JOIN arasındaki fark ve ne zaman hangisi (1.3)
- SQL: pencere fonksiyonu — ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...) (1.4a)
- SQL: pencere fonksiyonu ile yürüyen toplam / running total (1.4b)
- SQL: COALESCE / GREATEST / LEAST ile NULL ve aykırı değer yönetimi (1.5)
- Spark: SparkSession nedir, AQE ve shuffle.partitions ne işe yarar (2.1)
- Spark: transformation ile action farkı, lazy evaluation (2.2)
- Spark: şemayı el ile tanımlamak (StructType) ile inferSchema farkı, riski (2.3)
- Spark: broadcast join ne zaman ve neden kullanılır (2.4)
- Spark: UDF nedir, maliyeti nedir, ne zaman kabul edilebilir (2.5)
- Spark: küçük veri için ne zaman Spark yerine düz Python/pandas tercih edilir (2.6)
- Delta Lake: overwrite ile MERGE (upsert) arasındaki fark ve trade-off (2.7)
- Idempotency: bir pipeline'ın 'tekrar çalıştırılabilir' olması ne demek (2.8)
- Veri kalitesi: yazmadan önce ucuz assert/sanity check'lerin değeri (2.9)
- Medallion mimari: bronze / silver / gold katmanlarının görevi ne (genel)

16'sı da notsuz, akıcı çıkıyorsa hazırsın. Kaynak kod:
[notebooks/_deferred/page9-battery/12,13,14,15,16*.py](#) (EnergyLens repo).